

Implementar objetos de programabilidad con SQL

1. Conéctese a AdventureWorksLT

```
-- Verify key tables in AdventureWorksLT
```

```
SELECT TOP (5) CustomerID, FirstName, LastName  
FROM SalesLT.Customer;
```

```
SELECT TOP (5) SalesOrderID, OrderDate, CustomerID  
FROM SalesLT.SalesOrderHeader;
```

```
SELECT TOP (5) ProductID, Name, ListPrice  
FROM SalesLT.Product;
```

90 %	▼	✓ No se encontraron problemas.
Resultados	Mensajes	
CustomerID	FirstName	LastName
1	Orlando	Gee
2	Keith	Harris
3	Donna	Carreras
4	Janet	Gates
5	Lucy	Harrington
SalesOrderID	OrderDate	CustomerID
1	2008-06-01 00:00:00.000	29847
2	2008-06-01 00:00:00.000	30072
3	2008-06-01 00:00:00.000	30113
4	2008-06-01 00:00:00.000	29485
5	2008-06-01 00:00:00.000	29957
ProductID	Name	ListPrice
1	HL Road Frame - Black, 58	1431,50
2	HL Road Frame - Red, 58	1431,50
3	Sport-100 Helmet, Red	34,99
4	Sport-100 Helmet, Black	34,99
5	Mountain Bike Socks, M	9,50
✓ Consulta ejecutada correctamente.		

Cada consulta debe devolver hasta cinco filas de datos de muestra. El primer conjunto de resultados muestra los nombres de los clientes, el segundo muestra pedidos recientes con fechas y referencias de clientes, y el tercero enumera los productos con sus precios. Si alguna

consulta no devuelve filas o falla, confirme que la base de datos AdventureWorksLT esté restaurada correctamente y que tenga acceso de lectura.

2. Crear una vista para simplificar las consultas

```
CREATE OR ALTER VIEW SalesLT.vCustomerOrders AS
SELECT
    c.CustomerID,
    CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName,
    h.SalesOrderID,
    h.OrderDate
FROM SalesLT.Customer c
INNER JOIN SalesLT.SalesOrderHeader h ON c.CustomerID = h.CustomerID;
```

The screenshot shows a SQL Server Enterprise Manager window titled "SQLQuery1.sql...25 (sa (73))*". The query window contains the following SQL code:

```
1  
2  
3  
4  
5 SELECT TOP (5) *  
6 FROM SalesLT.vCustomerOrders  
7 ORDER BY OrderDate DESC;
```

Below the query window, the "Resultados" (Results) tab is active, displaying a table with 5 rows and 5 columns. The columns are CustomerID, CustomerName, SalesOrderID, and OrderDate. The data is as follows:

	CustomerID	CustomerName	SalesOrderID	OrderDate
1	29847	David Hodgson	71774	2008-06-01 00:00:00.000
2	30072	Andrea Thomsen	71776	2008-06-01 00:00:00.000
3	30113	Raja Venugopal	71780	2008-06-01 00:00:00.000
4	29485	Catherine Abel	71782	2008-06-01 00:00:00.000
5	29957	Kevin Liu	71783	2008-06-01 00:00:00.000

La consulta devuelve hasta cinco filas que muestran los pedidos más recientes. Cada fila incluye un CustomerID, el nombre completo del cliente, el SalesOrderID, y el OrderDate, lo que demuestra cómo la vista simplifica el acceso a los datos unidos del cliente y del pedido.

3. Crear un procedimiento almacenado para procesar un pedido

```
CREATE OR ALTER PROCEDURE dbo.AddOrderLineItem  
    @SalesOrderID INT,  
    @ProductID INT,  
    @Quantity INT  
AS
```

```

BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;

    -- Use Product ListPrice as UnitPrice
    DECLARE @UnitPrice DECIMAL(18,2);
    SELECT @UnitPrice = CAST(ListPrice AS DECIMAL(18,2))
    FROM SalesLT.Product
    WHERE ProductID = @ProductID;

    IF @UnitPrice IS NULL
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50010, 'Invalid ProductID specified.', 1;
    END

    -- Ensure SalesOrderID exists
    IF NOT EXISTS (SELECT 1 FROM SalesLT.SalesOrderHeader WHERE
SalesOrderID = @SalesOrderID)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50011, 'Invalid SalesOrderID specified.', 1;
    END

    -- Insert line item (no discount)
    INSERT INTO SalesLT.SalesOrderDetail (SalesOrderID, OrderQty,
ProductID, UnitPrice, UnitPriceDiscount)
    VALUES (@SalesOrderID, @Quantity, @ProductID, @UnitPrice, 0);

    -- Update header subtotal based on current line totals
    UPDATE h
    SET SubTotal = d.SumLineTotal,
        ModifiedDate = SYSUTCDATETIME()
    FROM SalesLT.SalesOrderHeader h
    INNER JOIN (
        SELECT SalesOrderID, SUM(LineTotal) AS SumLineTotal
        FROM SalesLT.SalesOrderDetail
        WHERE SalesOrderID = @SalesOrderID
        GROUP BY SalesOrderID
    ) d ON d.SalesOrderID = h.SalesOrderID;

```

```
COMMIT TRANSACTION;
```

```
END;
```

```
SQLQuery1.sql...25 (sa (73))* X
1  -- Add a line item to an existing order (choose a valid SalesOrderID)
2  DECLARE @SalesOrderID INT = (SELECT TOP 1 SalesOrderID
3    FROM SalesLT.SalesOrderHeader
4    ORDER BY SalesOrderID DESC);
5  EXEC dbo.AddOrderLineItem @SalesOrderID = @SalesOrderID,
6    @ProductID = 680,
7    @Quantity = 1; -- adjust ProductID as needed
8
9  SELECT TOP (5) *
10 FROM SalesLT.SalesOrderDetail
11 WHERE SalesOrderID = @SalesOrderID
12 ORDER BY SalesOrderDetailID DESC;
13
14 SELECT SalesOrderID, SubTotal, TaxAmt, Freight, TotalDue
15 FROM SalesLT.SalesOrderHeader
16 WHERE SalesOrderID = @SalesOrderID;
```

90 % ▼ ✖ 1 ⚠ 0 ↑ ↓

📄

Resultados

📧

Mensajes

	SalesOrderID	SalesOrderDetailID	OrderQty	ProductID	UnitPrice	UnitPriceDiscount	LineTotal	rowguid	ModifiedDate
1	71946	113407	1	680	1431.50	0,00	1431.500000	ECCEE64B-0AAE-48D4-817E-BE5F311E06C4	2026-09-22 10:52:12.320
2	71946	113406	1	916	31.584	0,00	31.584000	B571CEE0-A90A-4E76-9796-379B55352A28	2008-06-01 00:00:00.000

	SalesOrderID	Sub Total	TaxAmt	Freight	TotalDue
1	71946	1463.084	3.1163	0.9738	1467.1741

Este procedimiento almacenado realiza una inserción transaccional de un nuevo elemento de línea. Primero valida que el producto y el pedido existen: retrocede y arroja un error si alguno de ellos no es válido. Después de insertar la fila de detalles con el precio de lista del producto, recalcula el subtotal del pedido de todas las partidas y actualiza el encabezado. Envolver todo en una transacción garantiza que la operación sea atómica: o todos los cambios tienen éxito o no se aplica ninguno.

4. Crear una función escalar para cálculos reutilizables

SQLQuery1.sql...25 (sa (73))*

```
1 CREATE OR ALTER FUNCTION dbo.fnOrderTotal (@OrderID INT)
2 RETURNS DECIMAL(18,2)
3 AS
4 BEGIN
5     DECLARE @Total DECIMAL(18,2);
6
7     SELECT @Total = SUM(LineTotal)
8     FROM SalesLT.SalesOrderDetail
9     WHERE SalesOrderID = @OrderID;
10
11     RETURN ISNULL(@Total, 0.00);
12 END;
```

SQLQuery1.sql...25 (sa (73))*		
1	SELECT d.SalesOrderID, <u>dbo.fnOrderTotal(d.SalesOrderID)</u> AS OrderTotal	
2	FROM SalesLT.SalesOrderDetail d	
3	GROUP BY d.SalesOrderID	
4	ORDER BY d.SalesOrderID DESC;	

90 %	1	0	↑	↓
Resultados	Mensajes			
	SalesOrderID	OrderTotal		
1	71946	1463.08		
2	71938	74160.23		
3	71936	79589.62		
4	71935	5533.87		
5	71923	96.11		
6	71920	2527.13		
7	71917	37.76		
8	71915	1732.89		
9	71902	59894.21		
10	71899	1856.21		
11	71898	53248.69		
12	71897	10585.05		
13	71895	221.26		
14	71885	524.66		
15	71867	858.90		
16	71863	2777.14		
17	71858	11528.84		
18	71856	500.30		
19	71846	1884.39		

Cree una función escalar que devuelva el valor total de un pedido utilizando los totales de línea de AdventureWorksLT.

5. Crear una función con valores de tabla en línea (TVF)

```
CREATE OR ALTER FUNCTION dbo.GetCustomerOrders (@CustomerID INT)
RETURNS TABLE
AS
```

```

RETURN
(
    SELECT
        h.SalesOrderID,
        h.OrderDate
    FROM SalesLT.SalesOrderHeader h
    WHERE h.CustomerID = @CustomerID
);

```

1 |
2 | SELECT *
3 | FROM dbo.GetCustomerOrders(29929)
4 | ORDER BY OrderDate DESC;

90 % 2 0

Resultados Mensajes

	SalesOrderID	OrderDate
1	71902	2008-06-01 00:00:00.000

SQLQuery1.sql...25 (sa (73))*

1 |
2 | SELECT CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName, o.SalesOrderID, o.OrderDate
3 | FROM SalesLT.Customer c
4 | CROSS APPLY dbo.GetCustomerOrders(c.CustomerID) o
5 | WHERE c.CustomerID = 29929;

90 % 3 0

Resultados Mensajes

	CustomerName	SalesOrderID	OrderDate
1	Jeffrey Kurtz	71902	2008-06-01 00:00:00.000

Cree una TVF para devolver los pedidos de un cliente determinado en AdventureWorksLT. Las TVF son útiles en cláusulas SELECT y JOIN.

6. Crear un disparador para registrar cambios

-- Audit table

```

IF OBJECT_ID('dbo.OrderAudit') IS NULL
BEGIN
    CREATE TABLE dbo.OrderAudit (
        AuditID INT IDENTITY(1,1) PRIMARY KEY,
        OrderID INT NOT NULL,
        OldTotal DECIMAL(18,2) NULL,
        NewTotal DECIMAL(18,2) NULL,
        ChangedAt DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
    );
END
GO

```

-- Trigger on order details updates

```

CREATE OR ALTER TRIGGER SalesLT.trg_LogOrderTotalChange
ON SalesLT.SalesOrderDetail
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    ;WITH AffectedOrders AS (
        SELECT SalesOrderID FROM inserted
        UNION
        SELECT SalesOrderID FROM deleted
    ),
    -- New totals from the base table (already reflects changes)
    NewTotals AS (
        SELECT d.SalesOrderID, SUM(d.OrderQty * d.UnitPrice) AS Total
        FROM SalesLT.SalesOrderDetail d
        INNER JOIN AffectedOrders a ON d.SalesOrderID = a.SalesOrderID
        GROUP BY d.SalesOrderID
    ),
    -- Contribution of the newly inserted/updated rows
    InsertedTotals AS (
        SELECT SalesOrderID, SUM(OrderQty * UnitPrice) AS Total
        FROM inserted
        GROUP BY SalesOrderID
    ),
    -- Contribution of the previous row versions (empty on INSERT)
    DeletedTotals AS (
        SELECT SalesOrderID, SUM(OrderQty * UnitPrice) AS Total
        FROM deleted
        GROUP BY SalesOrderID
    )
    INSERT INTO dbo.OrderAudit (OrderID, OldTotal, NewTotal)
    SELECT
        n.SalesOrderID,
        n.Total - ISNULL(i.Total, 0) + ISNULL(d.Total, 0) AS OldTotal,
        n.Total AS NewTotal
    FROM NewTotals n
    LEFT JOIN InsertedTotals i ON n.SalesOrderID = i.SalesOrderID
    LEFT JOIN DeletedTotals d ON n.SalesOrderID = d.SalesOrderID;
END;

```

90 %   2  0  

 Resultados

 Mensajes

	AuditID	OrderID	OldTotal	NewTotal	ChangedAt
1	1	71946	1463.08	2926.17	2026-09-22 09:03:39.4683957

La declaración `SELECT` devuelve filas recientes de la tabla de auditoría. Cada fila muestra el `OrderID` afectado, el total anterior (`OldTotal`), el nuevo total (`NewTotal`) después del cambio de cantidad y una marca de tiempo. Esto confirma que el disparador registró automáticamente la modificación.